INVERTIS UNIVERSITY, BAREILLY

B.Tech CSE – Third Year

APPLIED DATA STRUCTURES & ALGORITHMS LAB (BCS553)

LAB 3

Assignment 3: Bubble Sort

Assignment aim: Sort arrays by comparing and swapping adjacent values.

Experiment No. 3.1 — Perform one bubble-sort pass

1. Experiment Title: Perform one bubble-sort pass

2. Aim: Implement the task specified in the BCS553 lab assignment.

3. Software / Tools Required: Java JDK, a text editor or IDE (such as IntelliJ IDEA, Eclipse, or VS Code), and a terminal/command prompt.

4. Theory: Bubble sort repeatedly compares neighbouring elements and swaps those out of order.

5. Algorithm / Procedure:
Make one left-to-right pass; compare adjacent values; swap when the left value is larger; print the resulting array.
Then compile and execute the Java program.

6. Program:

```
public class Task_01_Perform_one_bubble {
    public static void main(String[] args) {
        int[] a = {5, 1, 4, 2, 8};
        for (int i = 0; i < a.length - 1; i++) {
            if (a[i] > a[i + 1]) {
                int temp = a[i]; a[i] = a[i + 1]; a[i + 1] = temp;
            }
        }
        for (int value : a) System.out.println(value);
    }
}
```

7. Input: Fixed input values are declared in the program: the arrays specified in each Bubble Sort task.

8. Output:
1
4
2
5
8

Time / Space Complexity: O(n) time for one pass; O(1) auxiliary space.

9. Result: The program implements the specified task and produces the expected output for the given test data.

10. Viva-Voce Questions:

- Why compare neighbours?
- What is a swap variable used for?
- Is one pass enough to fully sort every array?

Experiment No. 3.2 — Sort an Array Using Bubble Sort

1. Experiment Title: Sort an array using bubble sort

2. Aim: Implement the task specified in the BCS553 lab assignment.

3. Software / Tools Required: Java JDK, a text editor or IDE (such as IntelliJ IDEA, Eclipse, or VS Code), and a terminal/command prompt.

4. Theory: Bubble sort repeatedly compares neighbouring elements and swaps those out of order.

5. Algorithm / Procedure:
Repeat adjacent comparisons over multiple passes; swap out-of-order neighbours; print the sorted array.
Then compile and execute the Java program.

6. Program:

```
public class Task_02_Sort_an_array_using_bubble_sort {
    public static void main(String[] args) {
        int[] a = {5, 1, 4, 2, 8};
        for (int pass = 0; pass < a.length - 1; pass++) {
            for (int i = 0; i < a.length - 1 - pass; i++) {
                if (a[i] > a[i + 1]) {
                    int temp = a[i]; a[i] = a[i + 1]; a[i + 1] = temp;
                }
            }
        }
        for (int value : a) System.out.println(value);
    }
}
```

7. Input: Fixed input values are declared in the program: the arrays specified in each Bubble Sort task.

8. Output:
1
2
4
5
8

Time / Space Complexity: $O(n^2)$ average/worst time; O(1) auxiliary space.

9. Result: The program implements the specified task and produces the expected output for the given test data.

10. Viva-Voce Questions:

- Why are two loops used?

- What does each pass accomplish?
- What is bubble sort's worst-case complexity?

Experiment No. 3.3 — Bubble sort three values

1. Experiment Title: Bubble sort three values

2. Aim: Implement the task specified in the BCS553 lab assignment.

3. Software / Tools Required: Java JDK, a text editor or IDE (such as IntelliJ IDEA, Eclipse, or VS Code), and a terminal/command prompt.

4. Theory: Bubble sort repeatedly compares neighbouring elements and swaps those out of order.

5. Algorithm / Procedure:
Run bubble-sort passes over the array; compare adjacent elements and swap if needed; print sorted values.
Then compile and execute the Java program.

6. Program:

```
public class Task_03_Bubble_sort_three_values{
    public static void main(String[] args) {
        int[] a = {9, 7, 3};
        for (int pass = 0; pass < a.length - 1; pass++) {
            for (int i = 0; i < a.length - 1 - pass; i++) {
                if (a[i] > a[i + 1]) {
                    int temp = a[i]; a[i] = a[i + 1]; a[i + 1] = temp;
                }
            }
        }
        for (int value : a) System.out.println(value);
    }
}
```

7. Input: Fixed input values are declared in the program: the arrays specified in each Bubble Sort task.

8. Output:
3
7
9

Time / Space Complexity: $O(n^2)$ average/worst time; O(1) auxiliary space.

9. Result: The program implements the specified task and produces the expected output for the given test data.

10. Viva-Voce Questions:

- Does bubble sort work on descending input?
- Is it in-place?
- What is meant by sorted ascending order?